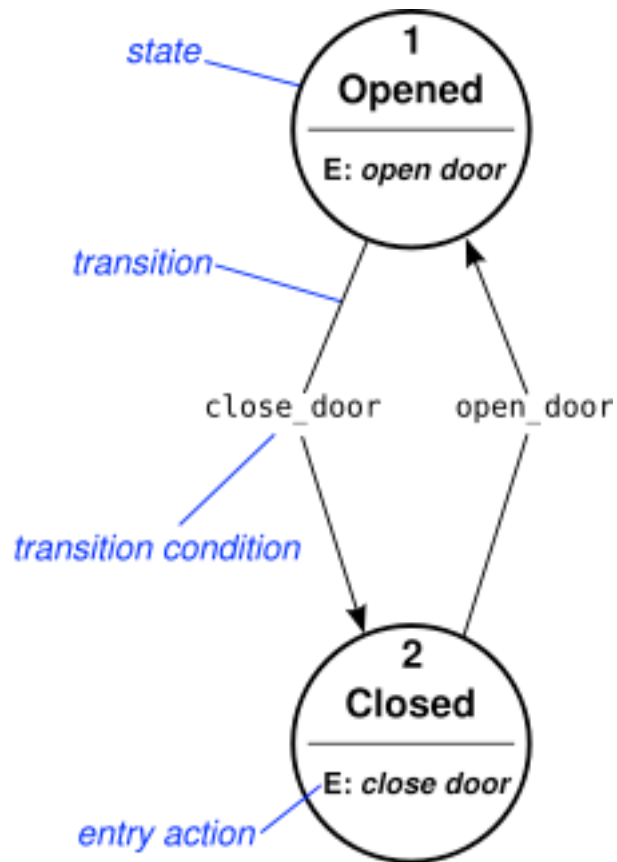


The State Pattern



Human Computer Interaction Research
University of Nevada, Reno



State Pattern

☐ Structural Patterns

- » adapter
- » decorator
- » façade
- » composite

☐ Creational Patterns

- » factory method
- » abstract factory
- » singleton

☐ Behavioral Patterns

- » strategy
- » observer
- » command
- » template method
- » iterator
- » **state**

Problem

Methods that have large, multipart conditional statements that depend on the object's state are difficult to maintain and extend

example

```
enum state{
    A,
    B,
    C
}
private state myState;

void doThis() {
    if(myState == A){
        // do that
    }
    else if(myState == B){
        state = A;
    }
    else if(myState == C){
        myState = A;
    }
}
```

```
void doThat() {
    if(myState == A) {
        // do that
    }
    else if(myState == B) {
        myState = A;
    }
    else if(myState == C) {
        myState = A;
    }
}
```

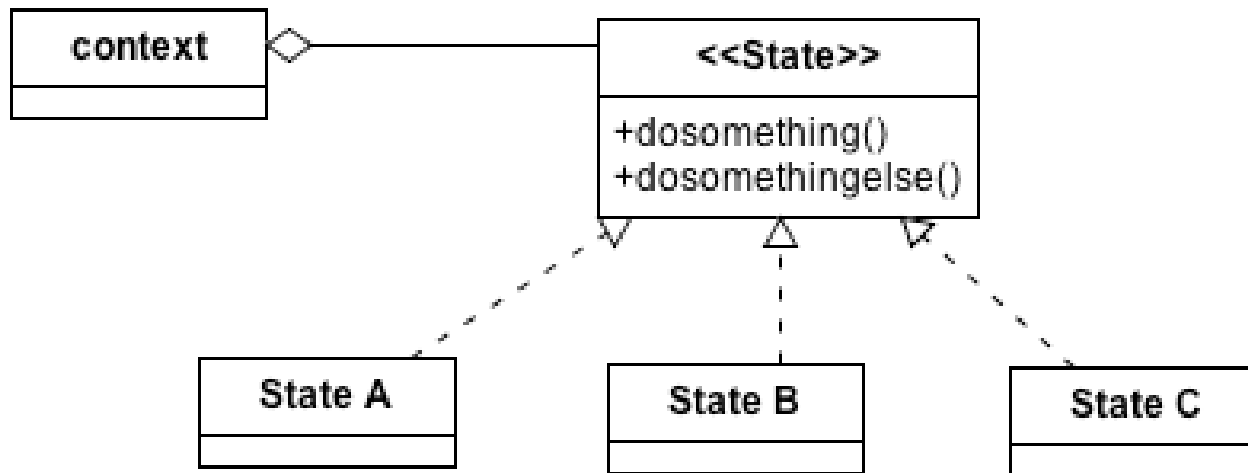
works but difficult to extend
for a new state

"Classes should be open for
extension, but closed for
modification"

State Pattern

The State Pattern allows an object to alter its behavior when its internal state changes. The object will appear to change its class.

Class diagram



Looks like
Strategy but
Intent is
different

State Interface

```
public interface State {  
    public void doThis();  
    public void doThat();  
}
```

State_A

```
public class State_A implements State {  
  
    StateContext statecontext;  
  
    public State_A(StateContext statecontext) {  
        this.statecontext=statecontext;  
    }  
  
    public void doThat() {  
        statecontext.setState(statecontext.getState_B());  
    }  
  
    public void doThis() {  
    }  
  
}
```


context

```
public class StateContext {  
    State State_A;  
    State State_B;  
    State myState;  
  
    public StateContext() {  
        State_A = new State_A(this);  
        State_B = new State_B(this);  
        myState = State_A;  
    }  
    public void setState(State newState) {  
        this.myState = newState;  
    }  
  
    public State getState_A() {  
        return State_A;  
    }  
  
    public State getState_B() {  
        return State_B;  
    }  
}
```

example



State Interface

```
public interface State {  
    public void open();  
    public void close();  
    public void lock();  
    public void unlock();  
    public void combinationEntered();  
    public void errorEntered();  
}
```

Context

```
public class GarageDoor {  
  
    State Opened;  
    State Locked;  
    State Closed;  
    State AwaitingCombination;  
    State state;  
  
    public GarageDoor() {  
        System.out.println("creating a garagedoor");  
        Opened= new Opened(this);  
        Closed= new Closed(this);  
        Locked = new Locked(this);  
        AwaitingCombination= new AwaitingCombination(this);  
        state=Closed;  
    }  
}
```

Context (cont.)

```
public void close() {  
    state.close();  
}
```

```
public void open() {  
    state.open();  
}
```

```
public void lock() {  
    state.lock();  
}
```

```
public void unlock() {  
    state.unlock();  
}
```

```
public void combinationEntered() {  
    state.combinationEntered();  
}
```

```
public void errorEntered() {  
    state.errorEntered();  
}
```

```
public void setState(State t) {  
    this.state=t;  
}
```

```
public State getOpened() {  
    return Opened;  
}
```

```
public State getLocked() {  
    return Locked;  
}
```

```
public State getClosed() {  
    return Closed;  
}
```

```
public State getAwaitingCombination() {  
    return AwaitingCombination;  
}
```

State_A

```
public class Opened implements State {

    GarageDoor door;

    public Opened(GarageDoor g) {
        this.door=g;
    }

    public void close() {
        System.out.println("Door closing");
        door.setState(door.getClosed());
    }

    public void combinationEntered() {
        System.out.println("Error: door not locked");
    }

    public void lock() {
        System.out.println("Error: can't lock as door is open");
    }

    public void open() {
        System.out.println("Error: door already open");
    }

    public void unlock() {
        System.out.println("Error: door already open");
    }

    public void errorEntered() {
        System.out.println("Error: door not locked");
    }

}
```

projector

desk

Group 1

Group 2

Group 3

Group 4

Group 5

Group 6

Group 7

Group 8

Group 9

Group 10

Group 11

Group 12

Group 13

Group 14

Group 15

Group 16

Group 17

Group 18

Group 19

Group 20

Group 21

Group 22

Group 23

Group 24

Group 25

Group 26

Group 27

Group 28

Group 29

pillar

skateboards

door